Real-Time Chat System Architecture (SignalR / WebSockets)

1. Overview

This document explains a scalable, production-ready architecture for a real-time chat system using SignalR/WebSockets, message brokers, and distributed state.

2. Core Components

• Client Layer

Web, Mobile, Desktop apps.

Uses SignalR/WebSocket client library for persistent connections.

• API Gateway

Routes chat requests, handles authentication tokens, enforces rate limits.

• Real-Time Hub (SignalR Server)

Maintains active connections.

Pushes messages instantly to groups/users.

Uses backplane for multi-instance scaling.

• Message Broker (Redis / Azure SignalR / Kafka)

Ensures real-time fan-out of messages across multiple chat servers.

Enables horizontal scaling.

• Chat Service Layer

Handles message validation, throttling, user presence, and metadata.

• Database Layer

Stores conversation history, user profiles, media references.

Supports pagination + indexing for fast query.

• Notification System

Sends push notifications (FCM/APNs) for offline users.

3. Data Flow

Client → SignalR Hub → Broker → Other Hub Servers → Connected Clients

Persisted messages → Database → Retrieved in chat history requests

4. Features Supported

• Typing indicators

• Online/offline presence

• Group chat

• Message delivery status

• File/attachment support

• Event-based system hooks

• Scalable multi-node architecture

5. Scalability Model

Single Server → Multi-Server with Redis → Global Load Balancing → Azure SignalR Service for elastic growth

6. Security Considerations

• JWT authentication on connection

• Message size limits

• Flood protection / rate limits

• Server-side validation

• Logging and monitoring hooks

This architecture can support enterprise-level chat workloads with minimal latency and high reliability.